aiPlat

aiPlat 知识体系技术白皮书

Ontology–Governed Compiled Knowledge System（OG–CKS）目标架构规范

目标架构规范 · 设计与治理基线

版本	v1.0 · 2026–11–11
性质	技术白皮书 / 目标架构规范（Normative）
状态	Proposed；今后设计与实现须以此为准，变更须走 ADR
范围	企业知识体系的模型、架构、数据、摄入、本体、事实、Wiki、索引、检索、安全、评测、运维与治理
重要说明	本文定义**目标架构与规范**，不是当前实现状态报告；当前代码是否满足，须另行审计。
诚实边界	不宣称 RAG accuracy；不把 Wiki 当事实源；不以“有白皮书”代替实现与验证。

阅读提示：本文定义目标架构与规范，不代表当前实现状态；实现差异须另行审计。本文不构成 accuracy/security 认证。

目录

01 摘要

02 1. 范围与定位

03 2. 核心命题

04 3. 术语

05 4. 设计原则（规范性）

06 5. 目标架构

07 6. 数据模型

08 7. 本体

09 8. 摄入

10 9. 事实库

11 10. Wiki

12 11. 索引

13 12. 检索

14 13. 回答

15 14. 质量与评测

16 15. 安全与多租户

17 16. 运维

18 17. 治理

19 18. 采用路线（非当前实现）

20 19. 风险与开放问题

21 20. 合规性要求清单

22 附录 A：查询流程

23 附录 B：Wiki 页面示例

24 附录 C：评测指标

25 附录 D：术语

26 结论

摘要

企业知识系统的核心矛盾不是“文档太多”，而是：

> **知识没有稳定的结构、没有可追溯的证据、没有可持续的维护机制。**

传统 RAG 每次查询都从原始文档重新检索、重新拼接，知识不会积累； 纯知识图谱需要大量人工建模，难以覆盖长尾； 纯 LLM Wiki 可读、可维护，但容易把合成内容误当事实。

本白皮书提出 **OG-CKS（Ontology-Governed Compiled Knowledge System，本体治理的编译式知识系统）**：

```
Raw sources（不可变事实源）
      +
Ontology（TBox：概念/关系/公理/约束）
      +
Fact Store（ABox：实体/关系/claim，带证据）
      ↓
Wiki（人类可读的编译视图，可审、可引用）
Indexes（FTS/Vector/Graph，可选、可重建）
      ↓
Knowledge Gateway（统一查询/融合/引用）
```

核心主张：

1. **两个真相层**：Raw sources + Ontology-typed Fact Store；
2. **Wiki 是读模型**，不是事实源；
3. **索引是派生**，可重建，不是知识；
4. **本体是 schema/约束**，不是一堆 YAML；
5. **Gateway 是编排器**，不是“三选一”；
6. **所有结论必须可引用；无证据则 abstain**；
7. **多租户隔离是硬边界**；
8. **知识质量必须可评测、可审计、可回滚。**

---

1. 范围与定位

1.1 目的

本白皮书用于：

- 统一知识体系的术语、模型、分层与职责；
- 规定不可回退的架构不变量；
- 指导新系统/新域/新接入方的设计；
- 作为评审、验收、审计与后续 ADR 的基线。

1.2 规范用语

用语	含义
MUST	强制；不满足即不合规
MUST NOT	禁止
SHOULD	强烈建议；例外需记录理由
MAY	可选

1.3 与当前实现的关系

本文**不描述**当前代码细节，也不假设当前实现已经满足。

- 当前实现是“候选实现”；
- 本文是“目标规范”；
- 差异必须通过 ADR/差距分析记录；
- 不允许用“当前代码如此”反推规范。

1.4 非目标

- 不替代领域专家；
- 不承诺完全自动本体学习；
- 不承诺零幻觉；
- 不替代业务验收/合规认证；
- 不规定必须使用某种数据库/向量库/图数据库；
- 不把“Wiki 可读”当成“知识正确”。

2. 核心命题

2.1 数据、知识、索引、视图不是一回事

概念	定义	是否事实源
Raw source	原始文档/数据，不可变	✅ 事实源
Ontology	概念/关系/公理/约束的 schema	✅ schema 权威
Fact	本体约束下的实体/关系/claim	✅ 事实权威
Wiki	人类可读的编译视图	❌ 派生
Index	FTS/Vector/Graph 检索结构	❌ 派生

2.2 两个真相层 + 两个视图

```
Truth 1: Raw sources
Truth 2: Ontology + Fact Store
View 1: Wiki
View 2: Indexes (FTS/Vector/Graph)
```

MUST：

- Raw 不可变；
- Fact 必须带来源；
- Wiki/Index 必须可重建；
- Wiki/Index 不得反向成为事实源。

2.3 本体治理，不是本体装饰

本体用于：

- 定义可接受的概念/关系；
- 约束抽取与校验；
- 支持实体解析；
- 支持查询路由；
- 支持矛盾检测；
- 支持 Wiki 结构。

MUST NOT：

- 只写 YAML 不建事实；
- 让 Wiki 自动反哺本体；

- 用 LLM 生成的本体直接当权威；
- 把本体当“第四个数据库”。

2.4 Wiki 是读模型

Wiki 的职责：

- 让人读懂；
- 让人审；
- 让人引用；
- 让人发现矛盾；
- 让答案可沉淀。

MUST NOT：

- 把 Wiki 当事实源；
- 让 Wiki 无引用发布；
- 让 LLM 无人审直接改 published Wiki；
- 用 Wiki 替代 raw/fact 的审计。

2.5 Gateway 是编排器，不是选择器

Gateway 负责：

- 租户/集合/权限；
- 查询理解；
- 路由；
- 多源融合；
- 冲突裁决；
- 引用；
- 降级披露。

MUST NOT：

- 让 Agent/UI/Pi 各自实现检索；
- 让 Gateway 在黑盒里“选一个库”；

- 把 Wiki、KG、向量当并列真相。

3. 术语

术语	定义
TBox	概念/关系/公理/约束的 schema
ABox	本体约束下的实体/关系/claim 实例
Fact	可验证的实体/关系/claim
Claim	一条可引用断言
Evidence	支持 claim 的 raw source + locator
WikiPage	人类可读的编译页面
Index	FTS/Vector/Graph 等检索结构
Tenant	数据隔离边界
Collection	租户内的知识集合
Revision	一次可追溯的变更
Review	人工审核状态
Abstain	证据不足时拒绝回答

4. 设计原则（规范性）

ID	原则	级别
P1	Raw sources 不可变	MUST
P2	每个 fact/claim 必须有 evidence	MUST
P3	Ontology 必须版本化	MUST
P4	Wiki/Index 必须可重建	MUST
P5	Wiki 不得作为事实源	MUST NOT
P6	索引不得作为事实源	MUST NOT
P7	所有入口必须经统一 Gateway	MUST
P8	租户/集合隔离必须可验证	MUST
P9	无证据必须 abstain	MUST
P10	低置信度必须人审	MUST
P11	变更必须可追溯/可回滚	MUST
P12	质量必须可评测	MUST
P13	小规模允许简单实现	SHOULD
P14	大规模允许替换存储/索引	MAY
P15	支持多模态/时序/不确定性	SHOULD

5. 目标架构

5.1 分层

```
┌──────────────────────────────────────────────────────────┐
│ 7. 治理：owner、版本、审核、评测、审计、claim boundary        │
├──────────────────────────────────────────────────────────┤
│ 6. 查询：KnowledgeGateway（route/merge/conflict/citations）  │
├──────────────────────────────────────────────────────────┤
│ 5. 视图：Wiki（人读） + Indexes（FTS/Vector/Graph，可重建）   │
├──────────────────────────────────────────────────────────┤
│ 4. 事实：Fact Store（实体/关系/claim + evidence）            │
├──────────────────────────────────────────────────────────┤
│ 3. 本体：TBox（类/关系/公理/状态/约束）                      │
├──────────────────────────────────────────────────────────┤
│ 2. 原始：Raw sources（不可变） + Source registry            │
├──────────────────────────────────────────────────────────┤
│ 1. 接入：API/Connector/Upload/Queue                      │
└──────────────────────────────────────────────────────────┘
```

5.2 写入路径

```
Source
  → Source registry（hash/tenant/license）
  → Parse/Chunk
  → Ontology-guided extraction
  → Fact candidates
  → Validation / Entity resolution
  → Human review（低置信/冲突）
  → Fact Store
  → Wiki compile
  → Index build
```

5.3 读取路径

```
Query
  → Gateway（tenant/collection/ACL）
  → Query understanding
  → Fact-first（ontology/structured）
  → Wiki（synthesis）
  → Index fallback（FTS/Vector）
  → Graph expansion（relation）
  → Fusion/Conflict/Citations
  → Answer or Abstain
```

5.4 关键不变量

ID	不变量
I1	Raw 内容不可原地修改
I2	Fact 必须有 source_ref/locator
I3	Ontology 有 version
I4	Wiki/Index 可重建
I5	跨租户查询 0 结果
I6	无 citation 不得 published
I7	Gateway 是唯一查询入口
I8	每次变更产生 revision
I9	冲突必须显式
I10	未知必须 abstain

6. 数据模型

6.1 Source

```
{
  "source_id": "src-...",
  "tenant_id": "t1",
  "collection_id": "c1",
  "uri": "s3://...",
  "sha256": "...",
  "media_type": "pdf|docx|html|md|image|video|api",
  "license": "...",
  "immutable": true,
  "ingested_at": "..."
}
```

规则：

- 不可变；
- hash 校验；
- 删除用 tombstone；
- 租户/集合 ACL；
- 来源许可记录。

6.2 Ontology

```
{
  "ontology_id": "customer-support",
  "version": "1.2.0",
  "classes": [],
  "relations": [],
  "axioms": [],
  "states": [],
  "constraints": []
}
```

规则：

- TBox 版本化；
- 变更迁移；
- 人审；
- 与 fact schema 对齐。

6.3 Entity / Relation / Claim

```
{
  "entity_id": "ent-...",
  "class_id": "Ticket",
  "properties": {},
```

```
  "tenant_id": "t1",
  "source_refs": [],
  "confidence": 0.92,
  "review_status": "reviewed"
}

{
  "relation_id": "rel-...",
  "from": "ent-...",
  "type": "assigned_to",
  "to": "ent-...",
  "source_refs": [],
  "confidence": 0.9,
  "valid_from": "...",
  "valid_to": null
}

{
  "claim_id": "cl-...",
  "text": "...",
  "subject": "ent-...",
  "predicate": "...",
  "object": "...",
  "evidence": [{"source_id": "src-...", "locator": "p.3"}],
  "confidence": "high",
  "review_status": "published"
}
```

6.4 WikiPage

```
{
  "page_id": "wiki-...",
  "tenant_id": "t1",
  "collection_id": "c1",
  "title": "...",
  "type": "entity|concept|comparison|overview|contradiction|answer",
  "status": "draft|reviewed|published|stale|deprecated",
  "sources": ["src-..."],
  "claims": ["cl-..."],
  "body_md": "...",
  "revision": 3,
  "reviewer": "...",
  "last_reviewed": "..."
}
```

6.5 Index

```
{
  "index_id": "...",
  "type": "fts|vector|graph",
  "model": "...",
  "version": "...",
  "source_revision": "...",
  "rebuildable": true
}
```

规则：

- 索引可重建；
- 记录 embedding 模型/版本；
- 索引不存储唯一事实。

7. 本体

7.1 TBox

必须定义：

- 类（Class）；
- 属性（Property）；
- 关系（Relation）；
- 公理（Axiom）；
- 状态机（State）；
- 约束（Constraint）；
- 版本（Version）。

7.2 本体构建

```
标准/专家种子
  → 代表性语料
  → 候选概念/关系（LLM/统计）
  → 聚类/归一
  → 人审
  → 公理/约束校验
  → 冻结 v1
  → 抽取 ABox
  → 监控漂移
  → 新版本
```

MUST NOT：

- 凭空定义无证据；
- 全自动发布；
- 用 Wiki 反哺本体；
- 无版本变更。

7.3 本体校验

- 类型合法性；
- 关系域/值域；
- 基数约束；
- 状态迁移；
- 矛盾检测；

- 孤立实体；
- 来源缺失。

7.4 本体演化

- 版本化；
- 迁移脚本；
- 向后兼容；
- 旧数据标记；
- 重建视图/索引；
- 变更审计。

---

8. 摄入

8.1 流程

9. Source registry；
10. 解析（PDF/Office/HTML/图片/视频/API）；
11. 分块；
12. 本体引导抽取；
13. 候选事实；
14. 校验/实体解析；
15. 人审；
16. 写入 Fact Store；
17. Wiki 编译；
18. 索引构建。

8.2 任务

MUST：

- 持久队列；

- 重试/死信；
- 幂等；
- 并发/背压；
- 进度/审计；
- 失败不丢 source。

8.3 人审

- 低置信；
- 冲突；
- 新类/新关系；
- 高风险领域；
- 发布前。

---

9. 事实库

9.1 职责

- 存实体/关系/claim；
- 带 evidence；
- 带时间/版本；
- 支持查询；
- 支持矛盾；
- 支持回滚。

9.2 证据

每个 fact/claim：

```
{
  "source_id": "src-...",
  "locator": "page=3|offset=...",
  "quote": "...",
  "confidence": "high|medium|low"
}
```

9.3 冲突

- 同一 subject/predicate 多 object；
- 时间/来源/置信度；
- 不强行合并；
- 显式 contradiction；
- 人审裁决。

9.4 时序

- `valid_from/valid_to`；
- 旧事实不删除，标失效；
- 支持“当时正确”。

---

10. Wiki

10.1 页面类型

- entity；
- concept；
- comparison；
- overview；
- contradiction；
- answer。

10.2 index.md / log.md

- `index.md`：内容目录；
- `log.md`：append-only 账本；
- 可 grep/CI/审计。

10.3 编译

```
Fact Store
  → 选择页面
  → 生成/更新正文
  → 插入 citations
  → 标记冲突/过期
```

```
→ 人审
→ published
```

10.4 规则

- 每个关键断言有 citation；
- 无 citation 不 published；
- Wiki 不写回 Fact Store；
- 答案可人审归档为 `answer` 页；
- Wiki 可重建。

---

11. 索引

11.1 类型

索引	用途
FTS	精确关键词/编号/中文
Vector	语义召回
Graph	关系/多跳

11.2 规则

- 可重建；
- 记录模型/版本；
- 租户隔离；
- 不作为事实源；
- 索引与事实版本对齐；
- 允许最终一致，但必须可检测滞后。

---

12. 检索

12.1 Gateway

唯一入口：

```
knowledge.query({
  tenant_id, collection_id, query,
  mode, top_k, require_citations
})
```

12.2 模式

模式	用途
structured	本体/事实查询
wiki	概念/综述
fts	精确关键词
vector	模糊召回
hybrid	默认组合
graph	多跳
auto	路由

12.3 顺序

```
1. ACL/tenant
2. Query understanding
3. Fact-first（结构化）
4. Wiki（综述）
5. FTS/Vector（兜底）
6. Graph（关系）
7. Fusion/Rerank
8. Conflict resolution
9. Citations
10. Answer/Abstain
```

12.4 冲突

```
raw > fact > wiki > index similarity
```

- 冲突显式返回；
- 不静默选一个；
- 无证据 abstain。

---

13. 回答

13.1 要求

- 每个关键断言有 citation；
- 明确“不知道”；
- 不把 Wiki 当原文；
- 不编造来源；

- 多租户不泄漏；
- 高风险领域加免责/升级。

13.2 输出

```
{
  "answer": "...",
  "citations": [],
  "route": [],
  "conflicts": [],
  "confidence": "high|medium|low",
  "abstained": false,
  "degraded": false
}
```

14. 质量与评测

14.1 指标

层	指标
本体	类/关系覆盖、实体解析 P/R、矛盾率
事实	证据覆盖、置信度校准、过期率
Wiki	citation 覆盖、stale 率、孤儿页
检索	Recall@k、MRR、context precision
回答	faithfulness、citation accuracy、abstain 正确率
隔离	跨租户 0 泄漏
运维	ingest 成功率、p95、重建耗时

14.2 评测集

- 每域 gold queries；
- 每域 gold facts/entities；
- 引用标注；
- 反例/不可回答；
- 多租户负向；
- 时序/冲突样本。

14.3 诚实锁

- 没有 live gold，不称 accuracy；
- 工程 smoke ≠ 质量；

- 评测结果绑定版本/配置/时间。

15. 安全与多租户

15.1 隔离

- tenant/collection 硬边界；
- 所有表/索引/对象带 tenant_id；
- 跨租户查询 0 结果；
- 共享 ontology 只共享 schema，不共享事实；
- 跨租户负向测试。

15.2 数据安全

- 加密 at rest/in transit；
- PII/secret 扫描；
- path allowlist；
- 上传限制；
- 审计；
- 保留/删除/法务 hold。

15.3 访问控制

- 角色/属性/对象级；
- Wiki 页面权限；
- Fact/Claim 权限；
- 导出权限；
- 审计所有访问。

16. 运维

16.1 SLO

- ingest 成功率/延迟；
- query p95；
- citation 命中率；
- freshness；
- 重建耗时；
- 隔离测试通过率。

16.2 存储

- 小规模：SQLite/FTS/Markdown；
- 大规模：PostgreSQL + PGVector/FAISS + 对象存储；
- 图存储可选：关系表/三元组/图数据库；
- 所有索引可重建。

16.3 版本

- ontology version；
- fact schema version；
- embedding model version；
- index version；
- Wiki revision；
- 兼容/迁移。

---

17. 治理

17.1 Owner

- 域 owner；
- 数据 owner；
- Wiki reviewer；

- security owner；
- platform owner。

17.2 变更

- RFC/ADR；
- 版本；
- 迁移；
- 回滚；
- 审计。

17.3 Claim

- 不允许无证据对外声明；
- 范围/版本/日期；
- 认证独立。

---

18. 采用路线（非当前实现）

Phase 0：基础

- Raw immutable；
- Source registry；
- 持久摄入；
- ontology v0；
- fact store；
- Wiki/Index 可重建；
- 单一 Gateway；
- 租户隔离。

Phase 1：Wiki-first

- 直接 source-to-wiki；

- index/log/lint；
- 引用；
- 人审；
- 答案归档。

Phase 2：本体/图谱

- TBox 版本化；
- ABox 抽取；
- 实体解析；
- GraphRAG；
- 冲突/时序。

Phase 3：规模化

- PG/PGVector；
- 多 worker；
- SLO；
- 多模态；
- 联邦知识；
- 认证/审计。

---

19. 风险与开放问题

风险	缓解
LLM 抽取错误	人审 + evidence + 校验
Wiki 幻觉	citation + review + lint
本体漂移	版本 + 迁移 + owner
租户泄漏	schema + 负向测试
索引滞后	版本 + 重建 + 检测
过度工程	小规模简单实现
成本	异步/缓存/按需
评估不足	gold + live + 审计

开放问题：

19. 本体自动学习的边界；

20. 多模态事实的表示；

21. 时序/不确定性标准；

22. 联邦知识治理；

23. 认证范围。

---

20. 合规性要求清单

一个实现只有满足以下，才可称“符合本白皮书”：

- [ ] Raw 不可变；
- [ ] Fact 有 evidence；
- [ ] Ontology 有版本；
- [ ] Wiki/Index 可重建；
- [ ] Wiki 不作为事实源；
- [ ] 单一 Gateway；
- [ ] 租户隔离负向测试；
- [ ] 无证据 abstain；
- [ ] 低置信人审；
- [ ] 变更可回滚；
- [ ] 质量可评测；
- [ ] claim 有边界。

附录 A：查询流程

```
Query
  → Gateway
  → ACL
  → Intent
  → Fact Store
  → Wiki
```

```
  → FTS/Vector
  → Graph
  → Fusion
  → Conflict
  → Citations
  → Answer / Abstain
```

附录 B：Wiki 页面示例

```
---
page_id: wiki-ticket-123
type: entity
tenant_id: t1
collection_id: support
status: published
sources: [src-1, src-2]
claims: [cl-1, cl-2]
reviewer: alice
last_reviewed: 2026-11-11
---

# Ticket 123

## 摘要
...

## 关键事实
- ... [src-1 p.3]
- ... [src-2 p.1]

## 冲突
- 来源 A 说 X；来源 B 说 Y。
```

附录 C：评测指标

指标	定义
Recall@k	相关证据是否在前 k
MRR	首个相关结果排名
Faithfulness	回答是否被证据支持
Citation accuracy	引用是否指向正确来源
Freshness	是否使用最新事实
Contradiction rate	未处理冲突比例
Isolation pass	跨租户 0 泄漏

附录 D：术语

术语	含义
TBox	schema
ABox	实例
Fact	可验证事实
Claim	可引用断言
Evidence	来源+定位
Wiki	可读视图
Index	检索结构
Gateway	统一查询入口

结论

本白皮书的核心不是“再建一个知识库”，而是建立一条可审计的知识生产链：

```
Raw → Ontology → Fact → Wiki → Index → Gateway → Citation → Answer
```

一句话：

> **本体定义“怎么理解”，事实库保存“什么是真的”，Wiki 负责“怎么读”，索引负责“怎么找”，Gateway 负责“怎么答且不越界”。**

---

aiPlat Knowledge System · Technical Whitepaper · v1.0 · 2026-11-11